

SALESFORCE DEVELOPER BRIDGE PROGRAM

Day 7 Documentation

Bulk Processing and Governor Limits

Project: Placement Management System
Day 7 of the Training Program

Student Name: Sudharshan Kuppili
PIN No.: 24PA5A0517

Table of Contents

1. Day 7 Objective
2. Business Scenario
3. Salesforce Governor Limits
4. SOQL Inside Loops
5. Bulkification
6. Lists, Sets and Maps
7. Existing PlacementApplicationTrigger
8. Bulk-Safe Duplicate Validation — validateNewApplications()
9. Bulk SOQL
10. DML Inside Loops
11. ApplicationService — handleApplicationUpdate()
12. Bulk Test — Five Placement Applications
13. Existing Application / Student / Job Identification
14. Duplicate Application Validation
15. Application Status Update — Selected
16. Related Student Status Update — Final Verification
17. Testing and Debugging
18. Test Results Summary
19. Debugging / Engineering Thinking
20. Interview Corner
21. Sprint Retrospective
22. Key Learnings
23. Conclusion

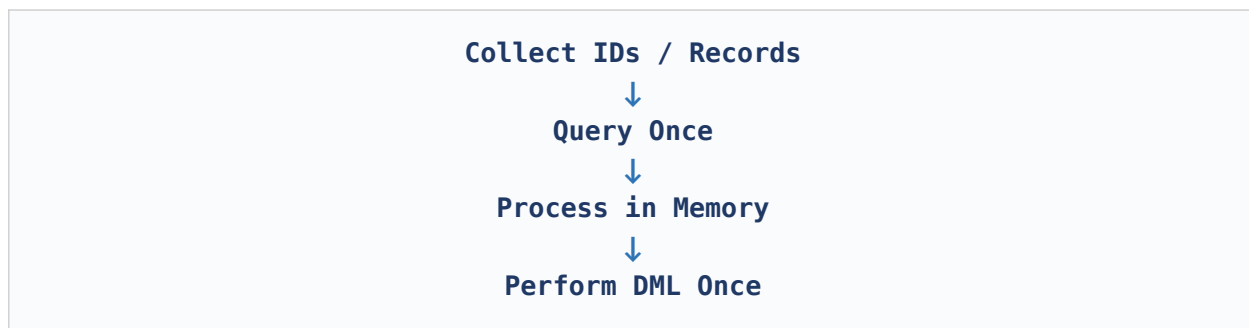
1. Day 7 Objective

Day 7 of the Salesforce Developer Bridge Program focused on Salesforce Governor Limits, bulkification, collection-based thinking, bulk SOQL, and bulk DML. The objective was to understand how to design Apex code that safely processes multiple records at once, rather than assuming only a single record will ever be handled.

Specifically, Day 7 covered:

- Understanding Salesforce Governor Limits
- Understanding bulkification
- Avoiding SOQL inside loops
- Avoiding DML inside loops
- Working with Lists, Sets and Maps for bulk-safe design
- Processing multiple records safely within a single transaction
- Testing bulk-safe Apex using Execute Anonymous

The core engineering pattern learned on Day 7 can be summarised as:



2. Business Scenario

The Placement Management System is designed to handle Placement Applications submitted by students against Jobs posted on the platform. In a real placement drive, Salesforce may need to process many Placement Application records within a single transaction — for example, during a bulk data import, a mass status update, or an automated trigger firing on multiple records at once.

Because Salesforce is a multi-tenant platform, Apex code must be designed to handle these situations safely. Code that queries or updates records one at a time inside a loop may work correctly for a single record but can fail when Salesforce processes a larger batch of records in the same transaction. This is why the Placement Management System's Apex logic — specifically the `ApplicationService` class and the `PlacementApplicationTrigger` — was built using bulk-safe, collection-based design from the start.

3. Salesforce Governor Limits

3.1 What Are Governor Limits?

Governor Limits are boundaries enforced by the Salesforce platform on the resources that a single transaction can consume, such as the number of database queries, the number of database write operations, CPU time, and memory (heap size).

3.2 Why Salesforce Enforces These Limits

Salesforce runs on shared, multi-tenant infrastructure. Governor Limits exist to ensure that no single transaction, org, or customer can consume a disproportionate share of shared platform resources, which protects the performance and stability of the platform for everyone.

3.3 Why Developers Must Design Carefully

Because these limits apply per-transaction, a piece of Apex code that works correctly when tested with a single record can still fail when Salesforce processes many records together in one transaction. Developers must therefore design Apex with bulk processing in mind from the very beginning, rather than treating it as an optimisation to add later.

3.4 Key Governor Limits Covered on Day 7

Resource	Synchronous Apex Limit
SOQL queries per transaction	100
SOQL records retrieved per transaction	50,000
DML statements per transaction	150
DML records processed per transaction	10,000
CPU time per transaction	10,000 ms
Heap size per transaction	6 MB

Main lesson: Governor Limits are engineering boundaries on shared Salesforce resources — they exist to encourage disciplined, bulk-safe design.

4. SOQL Inside Loops

A key lesson on Day 7 was understanding why placing a SOQL query inside a loop is dangerous. The following is an educational bad-pattern example used to illustrate the problem — it is not existing production code from the Placement Management System, but a pattern that must be avoided when writing Apex:

```
for (Placement_Application__c app : Trigger.new) {
    Student__c student = [
        SELECT Id, CGPA__c
        FROM Student__c
        WHERE Id = :app.Student__c
    ];
}
```

In this pattern, the SOQL query can execute once for every record in the loop. If `Trigger.new` contains 200 Placement Application records, this code could issue up to 200 separate queries in a single transaction, which exceeds the standard synchronous limit of 100 SOQL queries. The problem is not that the query itself is incorrect — it is that the query is executed once per record instead of once for the entire collection.

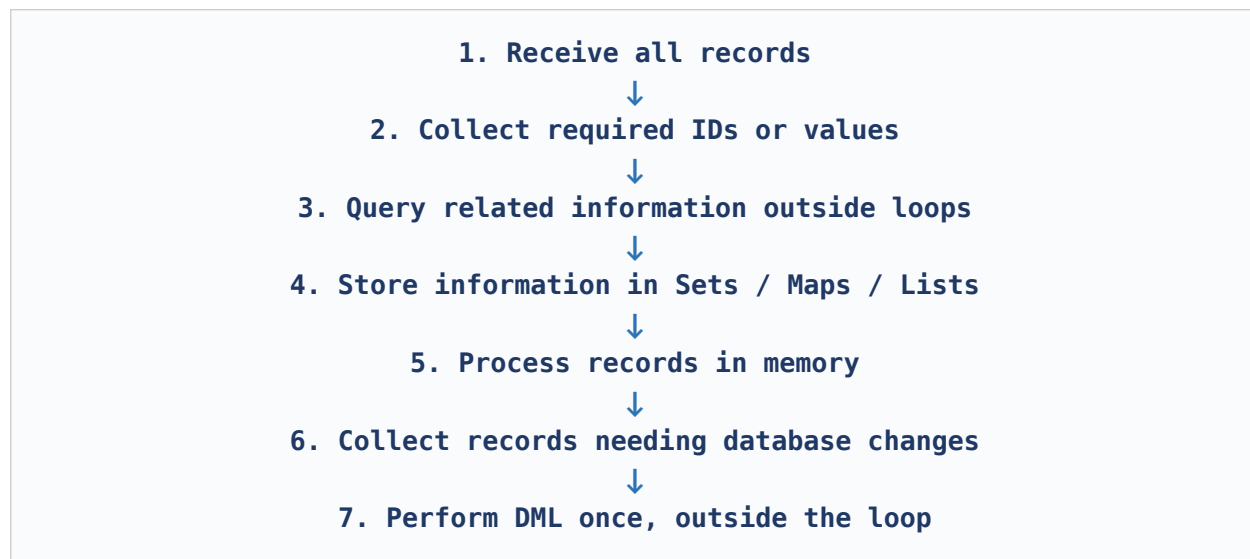
4.1 Record-by-Record vs. Collection-Based Processing

Record-by-record processing treats every incoming record as an isolated unit of work and repeats database operations for each one. Collection-based processing instead treats the entire batch of incoming records as a single unit of work: IDs are collected first, a single query retrieves all related records at once, and the results are processed in memory. This is the foundation of bulk-safe Apex design.

5. Bulkification

Bulkification means designing Apex so that the same logic safely handles 1 record, 10 records, or 200 records without changing its structure or exceeding Governor Limits. The goal is not to make the code more complex — it is to make the code scale safely.

The seven-step bulk-processing pattern learned on Day 7 is:



Following this pattern ensures that the same Apex method behaves identically and safely regardless of how many records Salesforce passes into it in a given transaction.

6. Lists, Sets and Maps

Collections are the core building blocks of bulk-safe Apex:

- List — an ordered group of records or values, used to hold records that will be processed or saved together.
- Set — a collection of unique values, used to remove duplicate IDs before running a query.
- Map — a key-based lookup structure, used to store retrieved records so they can be found quickly during in-memory processing without additional queries.

6.1 Set<Id> in ApplicationService.validateNewApplications()

The Placement Management System's ApplicationService class uses Set<Id> to collect Student and Job IDs from a batch of incoming applications:

```
Set<Id> studentIds = new Set<Id>();
Set<Id> jobIds = new Set<Id>();
```

Because a Set only stores unique values, duplicate Student or Job IDs supplied across multiple applications are automatically removed. This ensures that the subsequent SOQL query filters on the minimum necessary set of IDs, rather than querying the same record more than once.

7. Existing PlacementApplicationTrigger

The Placement Management System uses a single trigger on the Placement_Application__c object, which delegates all business logic to the ApplicationService class:

```
trigger PlacementApplicationTrigger
on Placement_Application__c (before insert, after update) {

    if (Trigger.isBefore && Trigger.isInsert) {
        ApplicationService.validateNewApplications(Trigger.new);
    }

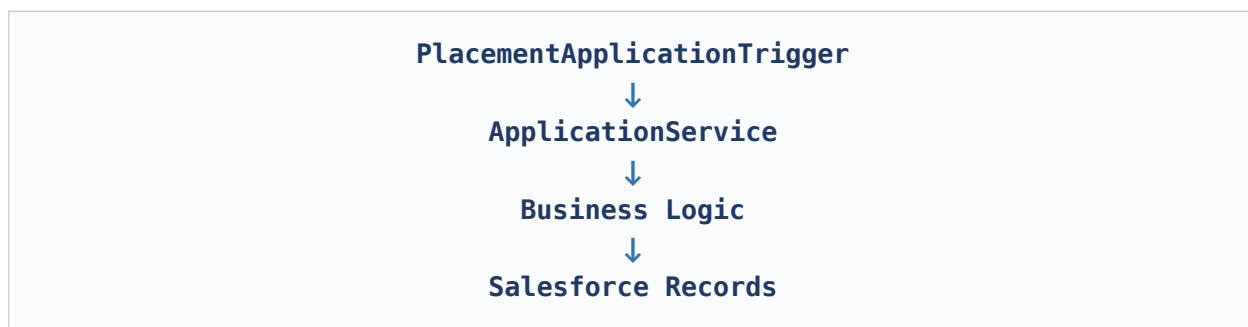
    if (Trigger.isAfter && Trigger.isUpdate) {
        ApplicationService.handleApplicationUpdate(
            Trigger.new,
            Trigger.oldMap
        );
    }
}
```

7.1 Key Concepts

- **Trigger.new** — the collection of new/incoming record versions being processed in the transaction.
- **Trigger.oldMap** — a map of the previous versions of the records, keyed by record Id, used to detect what changed.
- **Before Insert** — calls `ApplicationService.validateNewApplications(Trigger.new)` to block duplicate applications before they are saved.
- **After Update** — calls `ApplicationService.handleApplicationUpdate(Trigger.new, Trigger.oldMap)` to propagate a Selected status to the related Student record.

The Trigger itself contains no business logic — it only recognises the event and routes it to the appropriate method in ApplicationService. This keeps the Trigger lightweight and readable, and keeps all business rules centralised and testable in the service class.

7.2 Architecture Flow



8. Bulk-Safe Duplicate Validation — validateNewApplications()

The validateNewApplications() method in ApplicationService is responsible for preventing a student from applying to the same job more than once, and it is designed to work safely for any number of incoming applications:

- Collects Student IDs from all incoming applications into a Set<Id>.
- Collects Job IDs from all incoming applications into a Set<Id>.
- Runs one SOQL query to retrieve existing applications matching those Student and Job IDs.
- Builds a Set<String> of unique Student + Job combination keys from the existing applications.
- Checks each incoming application against the existing keys.
- Uses addError() to block any application that duplicates an existing Student + Job combination.

This design ensures that duplicate checking works correctly whether one application or many applications are being inserted in the same transaction, because the existing-application lookup is performed once, not once per record.

9. Bulk SOQL

Instead of querying for an existing application separately for each incoming record, the ApplicationService class retrieves all potentially matching applications in a single collection-based query:

```
List<Placement_Application__c> existingApplications = [
    SELECT Id, Student__c, Job__c
    FROM Placement_Application__c
    WHERE Student__c IN :studentIds
    AND Job__c IN :jobIds
];
```

By using the IN operator with the studentIds and jobIds Sets built earlier, Salesforce is asked a single question that covers every incoming application, instead of one question per application. This is the essence of bulk SOQL — supplying multiple IDs to one query rather than issuing multiple queries.

10. DML Inside Loops

Just as SOQL inside a loop is dangerous, so is performing a DML statement (insert, update, delete) inside a loop. The following bad pattern illustrates the risk:

```
for (Student__c student : students) {
    student.Placement_Status__c = 'Selected';
    update student;
}
```

With 200 Student records, this pattern could result in 200 separate DML statements in a single transaction, exceeding the standard synchronous limit of 150 DML statements.

The Placement Management System's implementation instead modifies each record in memory inside the loop, and performs a single update statement after the loop completes:

```
for (Student__c student : students) {
```

```

    student.Placement_Status__c = 'Selected';
}

update students;

```

The loop only changes field values in memory — no database operation occurs until the single `update students;` statement executes once, after the loop. This is bulk DML.

11. ApplicationService — handleApplicationUpdate()

The `handleApplicationUpdate()` method runs in the after-update context of the trigger and is responsible for propagating a Selected placement status from a Placement Application to its related Student record. It follows the same bulk-safe design:

- Collects Student IDs for applications whose `Status__c` changed to 'Selected' (comparing `Trigger.new` against `Trigger.oldMap`).
- Queries the related Student records once, using the collected IDs.
- Updates `Placement_Status__c` to 'Selected' on each Student record in memory.
- Performs a single bulk update on the Student records after processing.

The relevant update logic is:

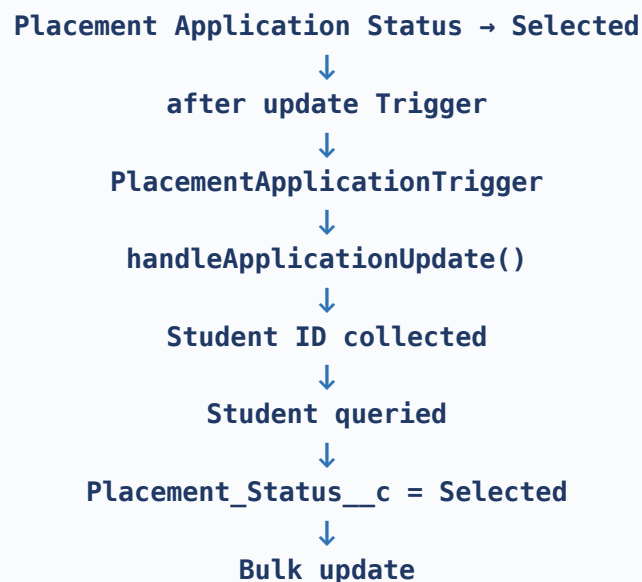
```

for (Student__c student : students) {
    student.Placement_Status__c = 'Selected';
}

update students;

```

11.1 Update Flow



12. Bulk Test — Five Placement Applications

A read-only Execute Anonymous test retrieved five Placement Application records in a single query and iterated over the resulting collection to display each Application ID together with its related Student and Job IDs.

Result:

Number of applications processed: 5

The debug log confirmed that all five applications were retrieved and processed as a single collection, with the Application, Student, and Job IDs displayed for each record. This demonstrates collection-based processing rather than record-by-record querying.

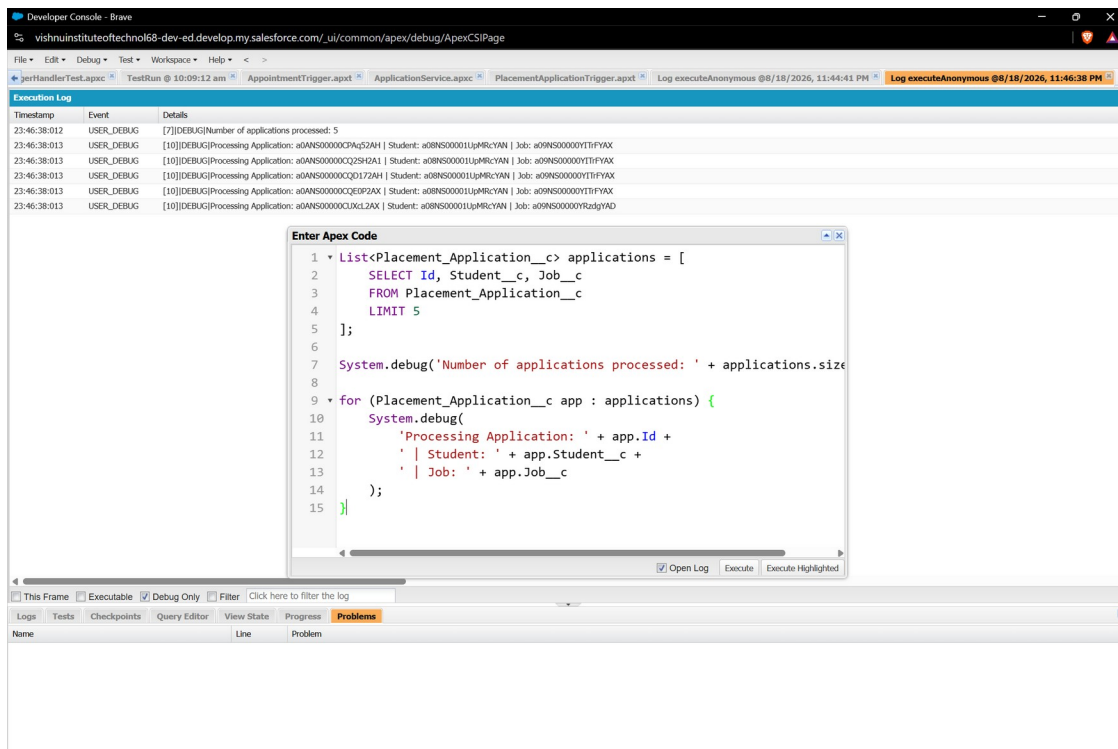


Figure 1: Bulk Processing Test with Five Placement Applications

13. Existing Application / Student / Job Identification

A read-only SOQL query was used to identify an existing Placement Application record with a non-null Student__c and Job__c, retrieving its Id along with the related Student ID and Job ID. These values were then used to construct a controlled duplicate-application test (Section 14).

The Execute Anonymous window below shows this identification step and the resulting duplicate-validation test executed together in the same debug log:

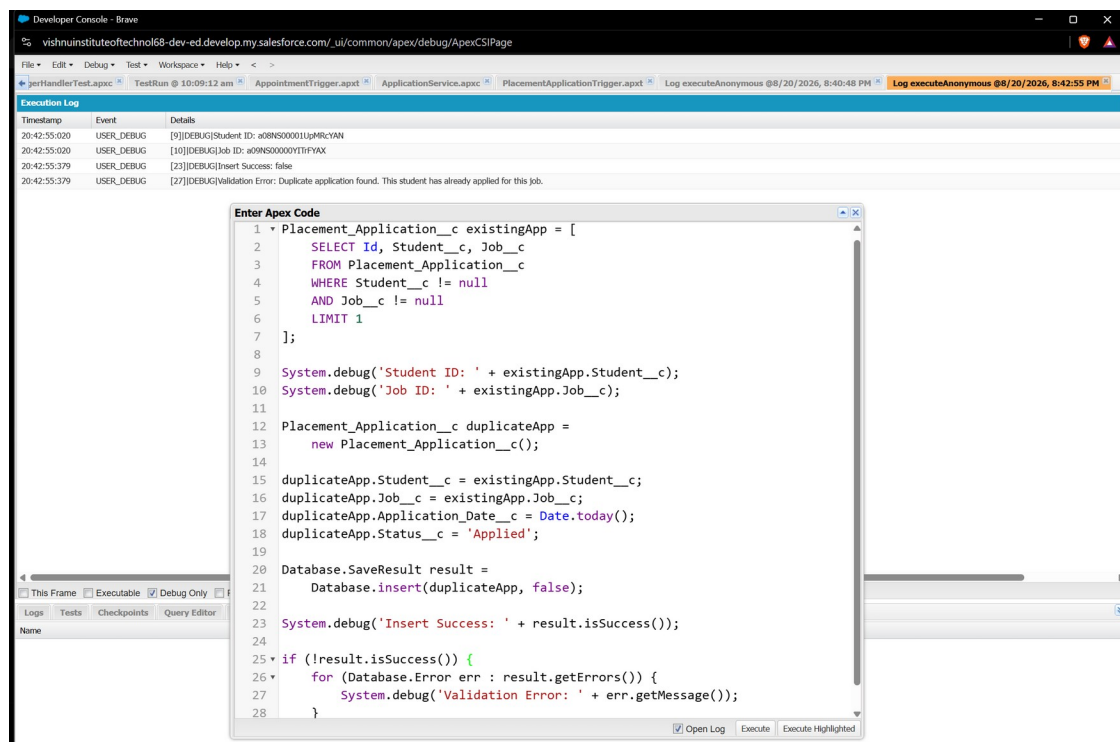


Figure 2: Existing Placement Application with Related Student and Job IDs, and Successful Duplicate Validation Test

14. Duplicate Application Validation

14.1 Earlier Attempt — Incorrect Job ID

An earlier duplicate-validation test used a manually typed Job Id that turned out to be invalid, which produced an incorrect-type error unrelated to the duplicate-validation logic being tested:

```

Insert Success: false
Validation Error: Job: id value of incorrect type: a09NS000000YItrFYAX
  
```

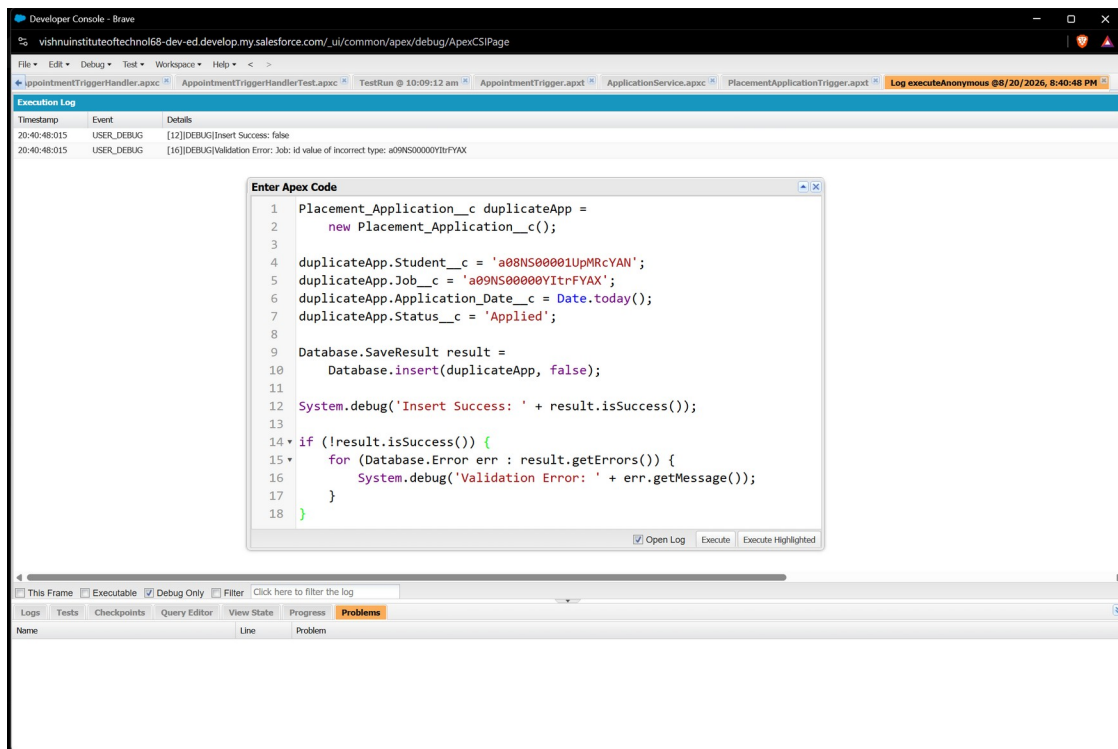


Figure 3: Earlier Attempt — Insert Blocked Due to an Incorrect Job ID (Not the Final Duplicate-Validation Result)

This earlier result does not demonstrate the duplicate-validation logic; it only shows that Salesforce rejected a malformed record Id. It is documented here for completeness but is superseded by the corrected test below.

14.2 Corrected Successful Test

The test was corrected by retrieving the Student and Job IDs directly from an existing Placement Application (Section 13) instead of typing them manually, then intentionally attempting to insert a duplicate application using the same Student + Job combination:

```

Placement_Application__c duplicateApp = new Placement_Application__c();
duplicateApp.Student__c = existingApp.Student__c;
duplicateApp.Job__c = existingApp.Job__c;
duplicateApp.Application_Date__c = Date.today();
duplicateApp.Status__c = 'Applied';

Database.SaveResult result =
    Database.insert(duplicateApp, false);

System.debug('Insert Success: ' + result.isSuccess());
  
```

Successful result:

```

Insert Success: false
Validation Error: Duplicate application found. This student has already applied
for this job.
  
```

This confirms that `validateNewApplications()` correctly detected the duplicate Student + Job combination and that `addError()` prevented the record from being inserted. The screenshot for this corrected, successful test is

shown in Figure 2 above, since both the identification query and the successful duplicate test were captured in the same Execute Anonymous log.

15. Application Status Update — Selected

The existing Placement Application identified earlier was updated so that its Status__c field was changed to 'Selected':

```
app.Status__c = 'Selected';
update app;
```

This update fired the after-update trigger context, which invoked ApplicationService.handleApplicationUpdate(). The debug log confirmed the status before the change, the related Student Id, and confirmation that the update completed successfully.

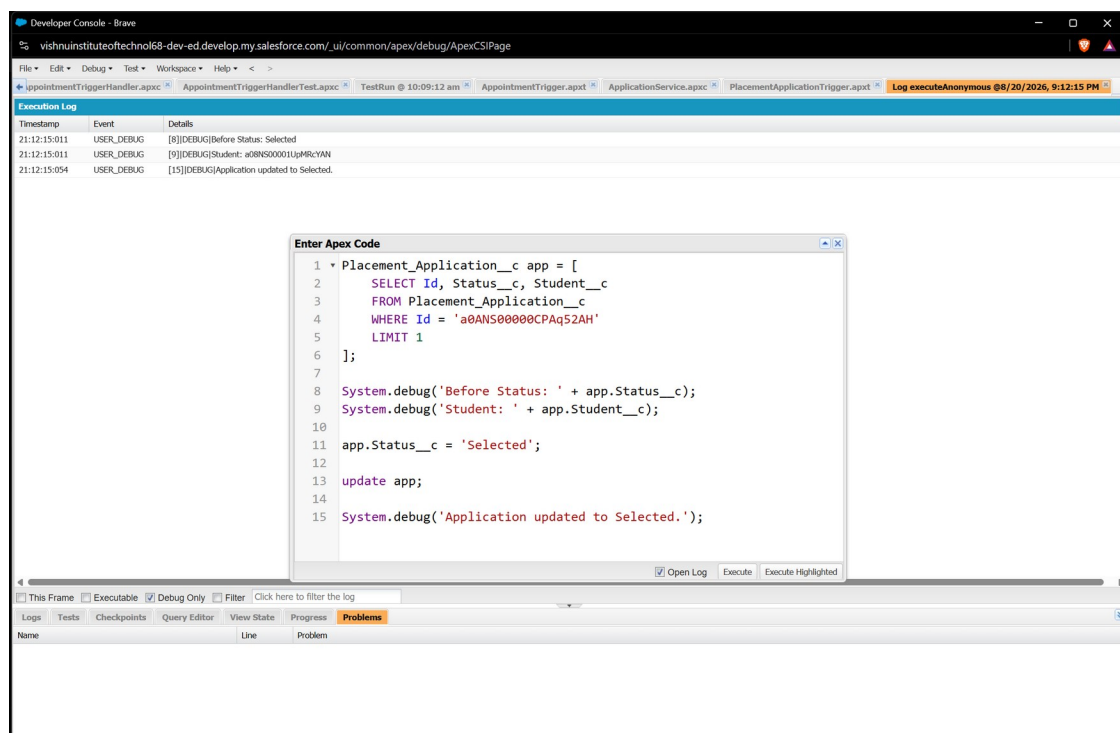


Figure 4: Placement Application Status Updated to Selected

16. Related Student Status Update — Final Verification

A read-only query was used to verify the related Student record after the Placement Application status was changed to Selected.

Final result:

```
Student Name: Sudharshan Kuppili
Placement Status: Selected
```

This confirms that the after-update trigger successfully invoked `handleApplicationUpdate()`, which located the related Student record, updated `Placement_Status__c` to 'Selected' in memory, and saved the change using a single bulk update statement.

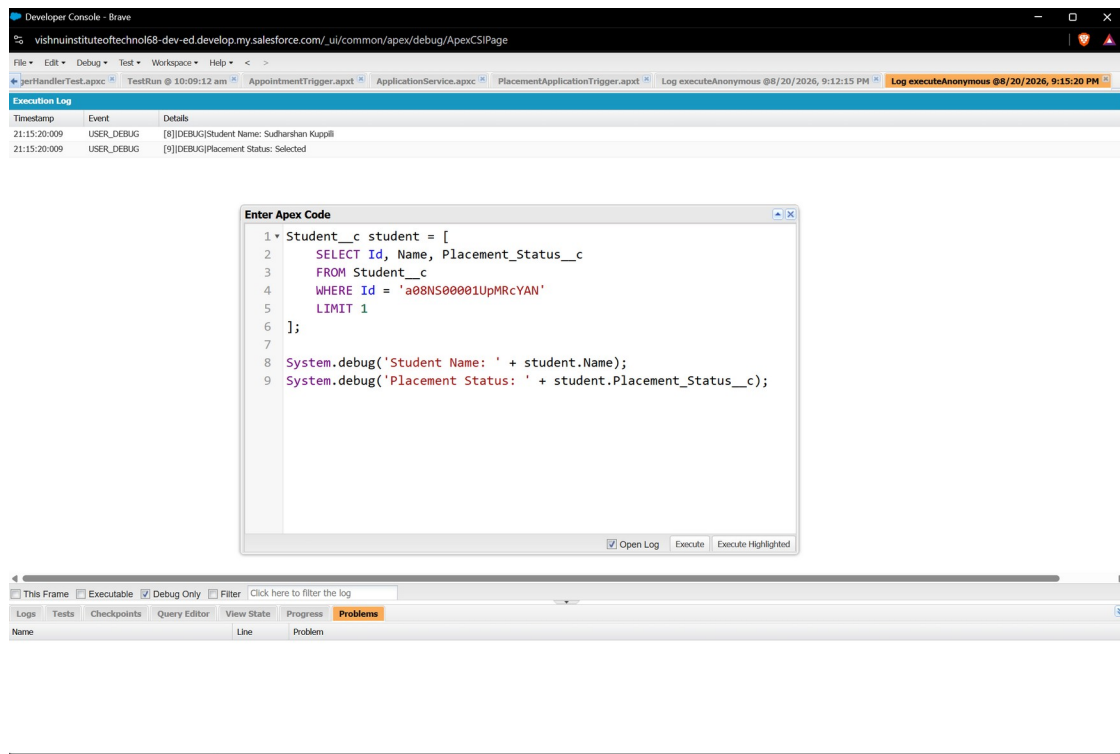


Figure 5: Final Verification of Student Placement Status

17. Testing and Debugging

Only tests that were actually completed during Day 7 are documented below.

Test 1 — Bulk Processing of Five Placement Applications

Expected: Five applications processed successfully as a collection.

Actual: Number of applications processed: 5

Status: PASSED

Test 2 — Existing Application Student/Job Identification

Expected: Existing Student and Job IDs retrieved from a valid Placement Application.

Actual: Student ID and Job ID successfully retrieved and logged.

Status: PASSED

Test 3 — Duplicate Application Validation

Expected: Duplicate application blocked by `addError()`.

Actual: Insert Success: false, with the message 'Duplicate application found. This student has already applied for this job.'

Status: PASSED

Test 4 — Application Status Update

Expected: Application Status__c changes to Selected and triggers after-update logic.

Actual: Status updated successfully; debug log confirmed 'Application updated to Selected.'

Status: PASSED

Test 5 — Related Student Status Update

Expected: Student Placement_Status__c becomes Selected.

Actual: Student Name: Sudharshan Kuppili, Placement Status: Selected.

Status: PASSED

18. Test Results Summary

Test	Expected Result	Actual Result	Status
Bulk processing of 5 applications	5 applications processed as a collection	Number of applications processed: 5	Passed
Existing application Student/Job identification	Existing Student and Job IDs retrieved	IDs successfully retrieved and logged	Passed
Duplicate application validation	Duplicate application blocked	Insert Success: false; validation error shown	Passed
Application status changed to Selected	Application Status__c updates to Selected	Status updated successfully	Passed
Related Student Placement Status updated to Selected	Student Placement_Status__c becomes Selected	Placement Status: Selected	Passed

19. Debugging / Engineering Thinking

Day 7 reinforced several practical engineering lessons that apply broadly to Salesforce development:

- Governor Limits are engineering boundaries on shared platform resources, not arbitrary restrictions.
- SOQL inside loops is dangerous because the number of queries scales with the number of records.
- DML inside loops is dangerous because the number of DML statements scales with the number of records.
- Trigger.new is a collection, not a single record — Apex should be designed around that assumption from the start.
- Set<Id> is useful for collecting unique IDs before querying.
- Lists are used to process and save groups of records together.
- Maps provide fast, in-memory record lookup after a single query.
- Queries should be moved outside loops wherever possible.
- Records should be processed in memory once related data has been retrieved.
- Bulk DML should be performed outside loops, as a single statement.

- Separating the Trigger from the ApplicationService class keeps the Trigger lightweight and the business logic centralised and testable.

20. Interview Corner

- 1. What are Salesforce Governor Limits?** Governor Limits are boundaries the Salesforce platform enforces on how many resources (queries, DML statements, CPU time, memory) a single transaction can use, since Salesforce is a shared, multi-tenant platform.
- 2. Why should SOQL not be placed inside loops?** Because the query would execute once per iteration; with many records this can quickly exceed the 100 SOQL queries per transaction limit.
- 3. Why should DML not be placed inside loops?** Because each iteration would trigger a separate DML statement; with many records this can exceed the 150 DML statements per transaction limit.
- 4. What is bulkification?** Designing Apex so that the same logic safely and correctly processes 1 record or many records without changing its structure or exceeding Governor Limits.
- 5. Why use Set<Id>?** A Set automatically removes duplicate IDs, ensuring a subsequent query only retrieves each related record once.
- 6. What is Trigger.new?** The collection of new/incoming versions of the records currently being processed by the trigger.
- 7. Why do we use collections in Apex?** Because Salesforce can pass multiple records into a single transaction; collections let logic process all of them together instead of one at a time.
- 8. What is bulk SOQL?** A single SOQL query that uses IN with a collection of IDs to retrieve all related records at once, instead of querying separately for each record.
- 9. What is bulk DML?** Performing a single DML statement (such as update) on a list of records, rather than issuing a separate DML statement per record.
- 10. How did your project prevent duplicate applications?** `validateNewApplications()` collects Student and Job IDs, runs one query for existing applications, builds a set of existing Student+Job keys, and calls `addError()` on any incoming application that matches an existing key.
- 11. How did the Application status update the related Student?** The after-update trigger calls `handleApplicationUpdate()`, which detects applications whose status changed to Selected, queries the related Students once, updates `Placement_Status__c` in memory, and performs a single bulk update.
- 12. Why is the Trigger kept lightweight?** So the Trigger remains easy to read and maintain; it only recognises the event context and delegates the actual business logic to the ApplicationService class.

21. Sprint Retrospective

What worked well?

Following the seven-step bulk-processing pattern (collect → query once → process in memory → DML once) made it straightforward to implement `validateNewApplications()` and `handleApplicationUpdate()` in a way that scales safely.

What felt natural?

Using `Set<Id>` to collect Student and Job IDs, and structuring the Trigger to simply delegate to `ApplicationService`, felt like a natural extension of the collection-based thinking introduced earlier in the program.

What was challenging?

Getting a valid Job Id for the duplicate-validation test was initially tricky — a manually typed Id caused an incorrect-type error. This was resolved by retrieving the IDs directly from an existing Placement Application rather than typing them by hand.

What was learned from Governor Limits?

Governor Limits are not just numbers to memorise for an exam — they are a practical review tool. Seeing a loop next to a SOQL query or a DML statement is now an immediate signal to check whether the logic can be restructured to query or update once, outside the loop.

What could be improved?

Additional test coverage using Apex test classes (rather than only Execute Anonymous) would give more confidence that the bulk-safe logic continues to work correctly as the Placement Management System grows.

How would this solution scale for many records?

Because `validateNewApplications()` and `handleApplicationUpdate()` both collect IDs, query once, and perform a single DML statement, the same logic should continue to work correctly whether the trigger receives 1, 50, or 200 Placement Application records in a single transaction.

22. Key Learnings

- Governor Limits are engineering boundaries on shared Salesforce resources.
- SOQL inside loops is dangerous.
- DML inside loops is dangerous.
- `Trigger.new` is a collection.
- Set is useful for unique IDs.
- Lists process groups of records.
- Maps can provide fast record lookup.
- Queries should be outside loops.
- Process records in memory.
- Perform bulk DML outside loops.
- Bulkification allows code to safely handle multiple records.

Final principle:

Collect First
↓
Query Once



23. Conclusion

Day 7 demonstrated the transition from individual-record thinking to collection-level thinking in Apex development. The Placement Management System's `ApplicationService` and `PlacementApplicationTrigger` were tested using multiple Placement Applications processed as a single collection, duplicate application validation was verified using a corrected, successful `Execute Anonymous` test, and the `Selected` status was confirmed to propagate correctly from a Placement Application to its related Student record through the after-update trigger and a single bulk DML statement.

This work demonstrated that the same bulk-safe Apex logic can process one record or many records without changing its structure — the core goal of bulkification.